A.Nowcoder Weekly Contest

按题意判断即可。

时间复杂度 $O(1)$。

```
void solve(){
    puts(read()>1599?"Unrated":"Rated");
}
```

B. ICPC Medal

显然最多执行 $log_2(n) = 30$ 轮操作，模拟每一轮操作即可。

时间复杂度 $O(log_2 n)$。

```
void solve(){
    ll a = read(), b = read(), c = read(), x = read(), y = read();
    while(1){
        ll t1 = c/x;
        b += t1;
        c -= t1 * x;
        ll t2 = b/y;
        if(t2 == 0) break;
        a += t2;
        b -= t2 *y;
        c += t2;
    }
    print(a);
}
```

C.Unique Number

显然第 i 个位置的操作次数不能大于 $\min\limits_{j=1}^{i-1} d_j$。

不妨依次将每个 d_i 设置为 $\min\limits_{j=1}^{i-1} d_j$。

为了使得不同的数字的个数最大，我们可以将 c_1 设为 d_1，后续每个数字设为

$$c_i = \begin{cases} c_{i-1} - 1 & c_{i-1} - 1 \le d_i \\ d_i & c_{i-1} - 1 > d_i \end{cases}$$

显然此时会使得数字呈现 5 4 2 1 的形态，为最多个数。

时间复杂度 $O(nlogn)$。

```
void solve(){
    ll n = read();
    vector<ll> c(n+1, INF);
```

```
    for(ll i=1;i<=n;i++) c[i] = min(read(), c[i-1]);
    ll pre = c[0];
    for(ll i=1;i<=n;i++){
        if(pre - 1 <= c[i] && pre >= 1){
            c[i] = pre-1;
        }
        pre = c[i];
    }
    set<ll> st;
    for(ll i=1;i<=n;i++) st.insert(c[i]);
    print(st.size());
}
```

D.Balanced Array

不难想到枚举每个 r 对应的最左边的 l，即滑动窗口。

不妨使用 set 维护 $\max\limits_{i=1}^{n} a_i$，$\min\limits_{i=1}^{n} a_i$ 。

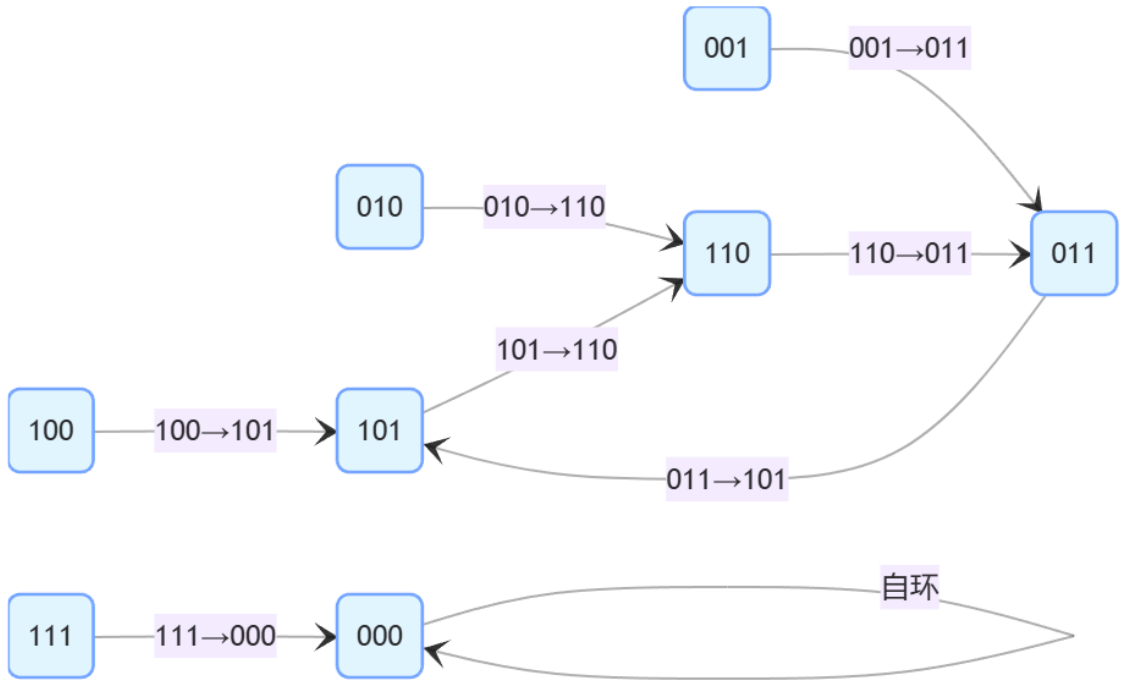
每次加入 a_r 后，不断删除 a_l 直到 $\max\limits_{i=1}^{n} a_i - \min\limits_{i=1}^{n} a_i \le 1$ 。则以 r 结尾的区间共有 $r-l+1$ 个。

时间复杂度 $O(nlogn)$。

```
void solve(){
    ll n = read();
    vector<ll> a(n+1);
    for(ll i=1;i<=n;i++) a[i] = read();
    multiset<ll> st;
    ll l = 1, ans = 0;
    for(ll r=1;r<=n;r++){
        st.insert(a[r]);
        while(l<r&&(*st.rbegin())-(*st.begin())>1){
            st.erase(st.find(a[l]));
            l++;
        }
        ans += r-l+1;
    }
    print(ans);
}
```

E.Maximize The Sum

对于 8 中情况，可以画出如下转换图。



观察发现，除了 000 和 111，其余所有情况最后都会进入三个中有两个 1 的循环中，且可以任意安排。

不妨猜测：

- 如果全 1，什么都不操作最优，答案为 n。
- 如果全 0，等效于无法操作，答案为 0。
- 其余所有情况一定可以转化到只剩一个 0 的情况，答案为 $n-1$。

时间复杂度 $O(n)$。

```
void solve(){
    ll n;cin>>n;
    string s;cin>>s;s = ' ' + s;
    vector<ll> a(n+1);
    for(ll i=1;i<=n;i++) a[i] = s[i] - '0';
    ll sum = accumulate(all(a), 0ll);
    if(sum == n) print(n);
    else if(sum == 0) print(0);
    else print(n-1);
}
```

F.Palindromic Value

显然最多有 n^2 个回文串。

可以使用枚举中心点并扩张的方法得到所有的字符串。

令 ls_r 为 所有以 r 为右端点的回文串的左端点， cnt_i 为前 i 个元素的分割方案数，sum_i 为前 i 个元素所有分割方案的价值和。

则：

$$cnt_r = \sum_{l \in ls_r} cnt_{l-1}$$

$$sum_r = \sum_{l \in ls_r} sum_{l-1} + cnt_{l-1} * (r - l + 1)^2$$

答案即为 sum_n。

时间复杂度 $O(n^2)$。

```
void solve(){
    ll n;cin>>n;
    string s;cin>>s;
    s = ' ' + s;
    vector<vector<ll>> ls(n+1);
    for(ll i=1;i<=n;i++){
        ls[i].pb(i);
        ll l = i-1, r = i+1;
        while(l>=1&&r<=n&&s[l]==s[r]){
            ls[r].pb(l);
            l--;
            r++;
        }
        l = i, r = i + 1;
        while(l>=1&&r<=n&&s[l]==s[r]){
            ls[r].pb(l);
            l--;
            r++;
        }
    }
    vector<ll> sum(n+1), cnt(n+1);
    cnt[0] = 1;
    for(ll i=1;i<=n;i++){
        for(auto l:ls[i]){
            sum[i] = (sum[i] + sum[l-1] + cnt[l-1]*(i-l+1)%MOD*(i-l+1)%MOD)%MOD;
            cnt[i] = (cnt[i] + cnt[l-1])%MOD;
        }

    }
    cout<<sum[n]<<'\n';
}
```

C++ 火车头

```
// FZANOTFOUND
#include <bits/stdc++.h>
```

```
using namespace std;

#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define ne " -> "
#define sep "======"
#define fastio ios::sync_with_stdio(false);cin.tie(0);
#define all(a) a.begin(), a.end()

typedef long long ll;
typedef unsigned long long ull;
typedef long double db;
typedef pair<long long,long long> PLL;
typedef tuple<ll,ll,ll> TLLL;
const ll INF = (ll)2e18+9;
//const ll MOD = 1000000007;
const ll MOD = 998244353;
const db PI = 3.14159265358979323;

//io functions
inline void rd(ll &x){x=0;short f=1;char c=getchar();while((c<'0'||c>'9')&&c!='-')
c=getchar();if(c=='-') f=-1,c=getchar();while(c>='0'&&c<='9') x=x*10+c-
'0',c=getchar();x*=f;}
inline ll read(){ll x=0;short f=1;char c=getchar();while((c<'0'||c>'9')&&c!='-')
c=getchar();if(c=='-') f=-1,c=getchar();while(c>='0'&&c<='9') x=x*10+c-
'0',c=getchar();x*=f;return x;}
inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9) pt(x/10);putchar(x%10+'0');}
inline void print(ll x){pt(x), puts("");}
inline void print(PLL x){pt(x.fi), putchar(' '), pt(x.se), putchar('\n');}
inline void print(vector<ll> &vec){for(const auto t:vec)pt(t),putchar(' ');puts("");}
inline void print(const map<ll, ll>& g) {for(const auto& [key, value]:g){cout<<"key: "
<<key<<ne<<value<<" ";}puts("");}
inline void print(vector<PLL> &vec){puts(sep);for(const auto v:vec){print(v);}puts(sep);}
inline void print(const map<ll, vector<ll>>& g) {for (const auto& [key, value] : g) { cout
<< "key: " << key << ne;for (const auto& v : value) {cout << v << " ";}cout << endl;}}

//fast pow
ll ksm(ll a, ll b=MOD-2, ll M=MOD){a%=M;ll res=1;while(b){if(b&1){res=(res*a)%M;}a=
(a*a)%M;b>>=1;}return res;}

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());//rng()
ull randint(ull l, ull r){uniform_int_distribution<unsigned long long> dist(l, r);return
dist(rng);}

void init(){

}

void solve(){

}
```

```
int main(){
    init();
    ll t = 1;
    fastio;cin>>t;
    //t = read();
    while(t--){
        solve();
    }
}
```